



**SIRIUS
SOFTWARE**

Back-end Challenge

**8-Ball Pool Match & Tournament Manager
In Ruby on Rails**

V1.0

LAUNCHING YOU INTO THE FUTURE

www.sirius.com.ar

Parque Empresarial Austral (Av. Sgto. Cayetano Beliera 3025, Pilar). Edificio M3.
administracion@sirius.com.ar

8-Ball Pool Match & Tournament Manager (Ruby on Rails Edition)

Overview

Your task is to build an **8-Ball Pool Match & Tournament Manager** using **Ruby on Rails**. The application should allow users to:

1. **Manage players**: create, read, update, and delete players.
2. **Schedule matches**: create, read, update, and delete matches.
3. **Prevent double-bookings** (two matches overlapping in time for the same player).
4. **Optionally handle tournaments or leagues** (e.g., track wins/losses, update rankings).
5. **Handle user profile pictures** using **Amazon S3 pre-signed URLs**.
6. **Use [Auth0](#) for authentication**: Users must log in/register via Auth0. Users are also players in the system.

The main objective is to learn how to:

- Set up a **Rails API-only** application.
- Use **Active Record** for database interactions.
- Work with **validations**, **associations**, and **transactions** to prevent data inconsistencies.
- Integrate **Amazon S3** for handling user profile pictures (uploading via pre-signed URLs).
- Implement **authentication and authorization using Auth0**.
- Implement **search and filtering** logic for players and matches.

Authentication with Auth0

Users must log in via **Auth0** before accessing the application. **A user in Auth0 corresponds to a player in the system.**

- On first login, the user is automatically registered as a player.
- Authenticated users can only modify their own player profile.
- Admin users (roles defined in Auth0) can manage all players and matches.

Required Environment Variables for Auth0:

- AUTH0_DOMAIN
 - AUTH0_CLIENT_ID
 - AUTH0_CLIENT_SECRET
 - AUTH0_AUDIENCE
-

Core Features

1. Player Management

A **Player** has (this is a suggestion, you can do any changes to it):

- **id** (primary key)
- **auth0_id** (string, required, unique, from Auth0)
- **name** (string, required)
- **ranking** (integer, optional — defaults to 0)
- **preferred_cue**
- **profile_picture_url** (**string, required**)
This field will store the final public URL or some reference to the file in S3.
Players must have a profile picture.

Suggested Rails Model

```
# app/models/player.rb
class Player < ApplicationRecord
  validates :auth0_id, presence: true, uniqueness: true
  validates :name, presence: true
  validates :ranking, numericality: { only_integer: true },
allow_nil: true
  validates :profile_picture_url, presence: true

  has_many :matches_as_player1, class_name: "Match", foreign_key:
"player1_id", dependent: :nullify
  has_many :matches_as_player2, class_name: "Match", foreign_key:
"player2_id", dependent: :nullify
end
```

Endpoints (Rails-Style)

- **POST /auth/register** – Registers a new user (handled via Auth0 callback, auto-creates a player).
- **POST /players** – Create a player manually (admin only).
- **GET /players** – List all players (admin only). Allow optional filtering by name.
- **GET /players/me** – Fetch the authenticated user's player details.
- **PATCH/PUT /players/me** – Update the authenticated user's player details.
- **DELETE /players/:id** – Delete a player (admin only).
- **Any other endpoints you consider would be good or necessary**

2. Match Scheduling

A **Match** has:

- **id** (primary key)
- **player1_id** (foreign key to **players.id**)
- **player2_id** (foreign key to **players.id**)
- **start_time** (datetime, required)
- **end_time** (datetime, optional — set when the match finishes)
- **winner_id** (foreign key to **players.id**, optional)
- **table_number** (integer, optional)

Endpoints (Rails-Style)

- **POST /matches** – Create a new match.
 - Must provide **player1_id**, **player2_id**, **start_time**.
 - Optionally provide **table_number**.
 - If double-booking is detected, respond with **409 Conflict**.
- **GET /matches** – List all matches.
 - Optional filters:
 - **date=YYYY-MM-DD** – returns matches on a specific date.
 - **status=upcoming|ongoing|completed** – filter by status.
- **GET /matches/:id** – Get details of a single match.
- **PATCH/PUT /matches/:id** – Update match details.
 - Can set **end_time**, **winner_id**, **table_number**, etc.
 - Also must check again for double-booking if **start_time** changes.
- **DELETE /matches/:id** – Delete a match.
 - Allowed only if the match has not started or if there's a special business logic reason.
- **Any other endpoints you consider would be good or necessary**

Player Availability & Concurrency

You **must prevent** a conflict where a single player is booked to play two matches simultaneously. For example:

- Match A is scheduled for Player A from 3:00 PM to 4:00 PM.
- A new match tries to schedule Player A from 3:30 PM to 4:15 PM → **overlap** → reject with 409 Conflict.

Basic Tournament or Ranking (Optional)

- Track each player's wins/losses.
- Update their **ranking** after each match result is posted.
- For instance, **ranking += 1** for each win.

You can do this in a background job or as part of the **Match** update flow when **winner_id** is assigned.

Searching & Filtering

Implement at least one:

1. **Filter matches by date**
Example: **GET /matches?date=2025-01-14**.
2. **Filter matches by status**
Example: **GET /matches?upcoming** or **ongoing** or **completed**
3. **Filter players by partial name**
Example: **GET /players?name=john**

Profile Pictures via Amazon S3 Pre-Signed URLs

Instead of directly uploading images to your Rails server, users will:

1. **Request a pre-signed upload URL** from your backend for S3.
2. **Upload the file** to that URL.
3. **Update the **player.profile_picture_url**** with the final S3 location.

Implementation Tips

- Use the `a` gem for generating pre-signed URLs.
 - Be careful with **IAM** and **S3 bucket permissions**. We will take that into account when correcting the challenge
-

Running the Project

In your `README.md`, include:

1. **Setup & Installation**
 - Ruby version, Gemfile dependencies, etc.
 - `rails new your_app_name --api` to generate an API-only Rails app.
2. **Database Setup**
 - Default SQLite, Postgres, or MySQL—your choice, but Postgres is commonly used.
 - Run migrations: `rails db:create db:migrate`.
3. **Environment Variables**
 - For S3: `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `S3_BUCKET`, `AWS_REGION`, etc.

Error Handling & Status Codes

- **400 Bad Request** – if inputs are invalid (e.g., missing required params).
- **409 Conflict** – if double-booking is detected or any other conflict arises.
- **404 Not Found** – if the resource doesn't exist.
- **500 Internal Server Error** – for unhandled exceptions.

Use Rails conventions to handle these, for example:

- Validate with Active Record, raise `ActiveRecord::RecordInvalid`, rescue in a global rescue block or `ApplicationController`.
 - Use `head :conflict` (409) or custom error classes when you detect overlaps.
-

Testing

Use **RSpec** (or Minitest) along with tools like **FactoryBot** and **Faker**:

1. **Player tests:**

- Creating players (valid and invalid).
 - Searching players by **name**.
 - Ensuring a profile picture is always present.
2. **Match tests:**
 - Creating matches without conflicts.
 - Preventing and rejecting overlapping matches (double-booking).
 - Updating matches and setting winners.
 3. **Profile Picture Flow** (mock S3 if you want).
 4. **Transaction/Concurrency tests** (if possible, to confirm no overlaps can occur).
-

Other points (you must do ALL of them)

- **Dockerize** your Rails app.
- Set up a **CI/CD pipeline** (GitHub Actions, CircleCI, etc.).
- Deploy the app to a cloud platform (AWS, Heroku, Render, etc.).
- **Swagger/OpenAPI** documentation.
- Anything else creative or helpful to your project's learning goals.

Learning Objectives

1. **Ruby on Rails (API-Only) Basics:** Setting up a Rails project, using Active Record, migrations, validations, etc.
 2. **Active Record Associations & Transactions:** Handling references between **Player** and **Match**, ensuring data integrity with transactions.
 3. **Concurrency & Scheduling Conflicts:** Implement real-world booking logic to prevent overlapping matches.
 4. **RESTful Endpoints & Error Handling:** Return proper HTTP status codes and messages for successful or failed operations.
 5. **Amazon S3 Integration:** Generate pre-signed URLs, upload images, and store the final image URL in your Rails models.
 6. **Filtering & Searching:** Implement search by partial name or date-based queries for matches.
-

Have fun building your Rails-based 8-Ball Pool Manager, and don't forget to handle those profile pictures via S3!